Unit 10: Pointers

CONTENTS

Objectives

After studying this unit, you will be able to:

- Discuss the concepts of pointers
- Identify pointer increment and scale factors
- Pointer expressions
- Pointers and arrays
- NULL Pointer

Introduction

Computers use their memory for storing instructions of the programs as well as the values of the variables. Since memory is a sequential collection of storage cells each cell has an address associated with it. Whenever we declare a variable, the system allocates, somewhere in the memory, a memory location and a unique address is assigned to this location. Whenever a value is assigned to this variable the value gets stored in the location having a unique address in the memory associated with that variable. Therefore, the values stored in memory can be manipulated using their addresses. Pointer is an extremely powerful mechanism to write efficient programs.

Incidentally, this feature makes C stand out as the most powerful programming language. Pointers are the topic of this unit.

10.1 Pointers

A memory variable is merely a symbolic reference given to a memory location. Now let us consider that an expression in a C program is as follows:

int a = 10, b = 5, c;

c = a + b;

The above expression implies that a, b and c are the variables which can hold the integer data. Now from the above mentioned statement let us assume that the variable 'a' occupies the address 3000 in the memory, 'b' occupies 3020 and the variable 'c' occupies 3040 in the memory. Then the compiler will generate the machine instruction to transfer the data from the location 3000 and 3020 into the CPU, add them and transfer the result to the location 3040 referenced as c. Hence

we can conclude that every variable holds two values:

Address of the variable in the memory (l-value)

Value stored at that memory location referenced by the variable. (r-value)

Pointer is nothing but a simple data type in C programming language, which has a specialcharacteristic to hold the address of some other memory location as its r-value. C programminglanguage provides '&' operator to extract the address of any object. These addresses can be storedin the pointer variable and can be manipulated.

The syntax for declaring a pointer variable is,

<data type> *<identifier>;

Example

int n;

int *ptr; /* pointer to an integer*/

The following statement assigns the address location of the variable n to ptr, and ptr is a pointer to n.

ptr=&n;

Since a pointer variable points to a location, the content of that location is obtained by prefixing the pointer variable by the unary operator * (also called the indirection or dereferencing operator) like, *<pointer_variable>.

: # include<stdio.h>

main()

{

int a=10, *ptr;

ptr=&a; /* ptr points to the location of a */

printf("The value of a pointed by the pointer ptr is: %d", *ptr);

/* printing the value of a pointed by ptr through the pointer ptr*/

}

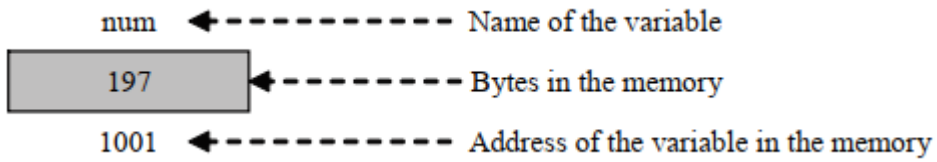
A null value can be assigned to a pointer when it does not point to any data or in the other words, as a good programming habit every pointer should be initialized with the null value. A pointer with a null value assigned to it is nothing but a pointer which contains the address zero.

The precedence of the unary operators '&' and '*' are same in C language. Here as a special case we can mention that '&' operator cannot be used or applied to any arithmetic expression, it can only be used with an operand which has unique address.

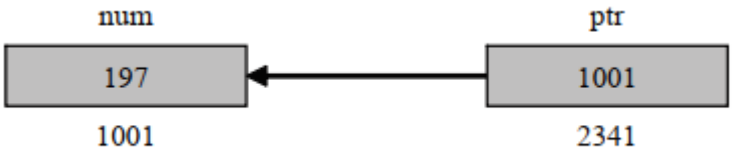
Pointer is a variable which can hold the address of a memory location. The value stored in a pointer type variable is interpreted as an address. Consider the following declarative statement:

intnum = 197;

This statement instructs the compiler to reserve a 2-byte memory location (assuming that the target machine stores an int type in two bytes) and to put the value 84 in that location. Assume that a system allocates memory location 1001 for num. diagrammatically it can be shown as:



As the memory addresses are numbers, they can be assigned to some other variable. Let ptr be the variable which holds the address of variable num. We can access the value of num by the variable ptr. Thus, we can say "ptr points to num". Diagrammatically, it can be shown as:



10.2 Accessing the Address of a Variable

The actual location of a variable in the memory is system dependent and therefore, the address of a variable is not known to us immediately. How can we then determine the address of a variable?

This can be done with the help of the operator & available in C. The operator & immediately preceding a variable return the address of the variable associated with it.

Example: The statement

P = &quantity;

Would assign the address 5000 to the variable p. The & operator can be remembered as 'address of'.

The & operator can be used only with a simple variable or an array element. The following are illegal use of address operator:

& 125 (pointing at constant).

Intx[10];

&x (pointing at array names).

&(x+y) (pointing at expressions).

If x is an array, then expression such as

&x[0] and &x[i+3]

are valid and represent the addresses of 0th and (i+3)th elements of x

10.3 Pointer Declaration

Since pointer variables contain address that belongs to a separate data type, they must be declared as pointers before we use them. Pointers can be declared just a any other variables. The declaration of a pointer variable takes the following form:

data_type *pt_name;

The above statement tells the compiler three things about the variable pt_name.

1. The asterisk (*) tells that the variable pt_name is a pointer variable.

2. pt_name needs a memory location.

3. pt_name points to a variable of type data type.

 : The statement

int *p;

declares the variable p as a pointer variable that points to an integer data type (int). The type int refers to the data type of the variable being pointed to by p and not the type of the value of the pointer.

Given below are some more examples of pointer declaration

Pointer declaration	Interpretation
Int *roll number;	Create a pointer variable roll number capable of pointing to an integer type vari able or capable of holding the address of an integer type variable
char *name;	Create a pointer variable name capable of pointing to a character type variable or capable of holding the address of a character type variable
float *salary;	Create a pointer variable salary capable of pointing to a float type variable or capable of holding the address of a float type variable

10.4 Address Operator - &

Once a pointer variable has been declared, it can be made to point to a variable by assigning the address of that variable to the pointer variable. The address of a variable can be extracted using address operator - &.

An expression having & operator generates the address of the variable it precedes. Thus, for example,

&num

produces the address of the variable num in the memory. This address can be assigned to any pointer variable of appropriate type (i.e., the data type of variable num) using an assignment statement such as p = # which causes p to point to num. That is, p now contains the address of num.

The assignment shown above is known as pointer initialization. Before a pointer is initialized, it should not be used. A pointer variable can be initialized in its declaration itself.

int x;

int *p = &x;

statement declares x as an integer variable and p as a pointer variable and then initializes p to the address of x. This is an initialization of p, not *p. On the contrary, the statement

int *p = &x, x;

is invalid because the target variable x is not declared before the pointer.

10.5 Indirection Operation - *

Since a pointer type variable contains an assigned address of another variable the value stored in the target variable can be obtained using this address. The value store in a variable can be referred to using a pointer variable pointing to this variable using indirection operator (*).

Example: Consider the following code.

int x = 109;

int *p;

p = &x;

Then the following expression

*p

Represents the value 109.

10.6 Pointer Variables

The actual address of a variable is not known immediately. We can determine the address of a variable using 'address of' operator (&). We have already seen the use of 'address of' operator in the scanf( ) function.

Another pointer operator available in C is "*" called "value a address" operator. It gives the value stored at a particular address. This operator is also known as 'indirection operator'.

 *Example*:

main( )

{

inti = 3;

printf ("\n Address of i: = %u", &i); /* returns the address * /

printf ("\t value i = %d", * (&i)); /* returns the value of address of i */

}

10.7 Initialization of Pointer Variables

Since pointer variables contain address that belong to a separate data type, they must be declared as pointers before we use them.

The declaration of a pointer variable takes the following form:

data_type *pt_name

This tells the compiler three things about the variable pt_name.

1. The asterisk (*) tells that the variable pt_name is a pointer variable.

2. pt_name needs a memory location.

3. pt_name points to a variable of type data type.

Example: int *p; declares the variable p as a pointer variable that points to an integer data type. The type int refers to the data type of the variable being pointed to by p and not the type of the value of the pointer.

Once a pointer variable has been declared, it can be made to point to a variable using an assignment statement such as p = &quantity; which causes p to point to quantity. That is, p now contains the address of quantity. This is known as pointer initialization. Before a pointer is initialized, it should not be used. A pointer variable can be initialized in its declaration itself.

Example: int x, *p=&x; statement declares x as an integer variable and p as a pointer variable and then initializes p to the address of x. This is an initialization of p, not *p. On the contrary, the statement int *p = &x, x; is invalid because the target variable x is declared first.

10.8 Accessing a Variable through its Pointer

Consider the following statements:

int q, * i, n;

q = 35;

i = & q;

n = * i;

i is a pointer to an integer containing the address of q. In the fourth statement we have assignedthe value at address contained in i to another variable n. Thus, indirectly we have accessed thevariable q through n. using pointer variable i.

10.9 Pointer Expression

Like other variables, pointer variables can be used in expressions. Arithmetic and comparison operations can be performed on the pointers. For example, if p1 and p2 are properly declared and initialized pointers, then following statements are valid.

y = *p1 * *p2; /multiply values stored in variables pointed to by *p1/and *p2

sum = sum + *p1; /increment sum by the value stored in the variable/pointed to by p1

The pointer may point to any location in the memory therefore you should be careful while using pointers in your programs.

10.10 Pointer arithmetic

A pointer in c is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -

To understand pointer arithmetic, let us consider that ptr is an integer pointer which points to the address 1000. Assuming 32-bit integers, let us perform the following arithmetic operation on the pointer

Following arithmetic operations are possible on the pointer in C language:

Increment

Decrement

Addition

Subtraction

Comparison

Increment:

It is a condition that also comes under addition. When a pointer is incremented, it actually increments by the number equal to the size of the data type for which it is a pointer.

Example:

If an integer pointer that stores address 1000 is incremented, then it will increment by 2(size of an int) and the new address it will points to 1002. While if a float type pointer is incremented then it will increment by 4(size of a float) and the new address will be 1004.

Decrement:

It is a condition that also comes under subtraction. When a pointer is decremented, it actually decrements by the number equal to the size of the data type for which it is a pointer.

For Example:

If an integer pointer that stores address 1000 is decremented, then it will decrement by 2(size of an int) and the new address it will points to 998. While if a float type pointer is decremented then it will decrement by 4(size of a float) and the new address will be 996.

Example:

Program to illustrate pointer increment/decrement

```
#include <stdio.h>

// Driver Code
int main()
{
        // Integer variable
        int N = 4;

        // Pointer to an integer
        int *ptr1, *ptr2;

        // Pointer stores
        // the address of N
        ptr1 = &N;
        ptr2 = &N;

        printf("Pointer ptr1 "
                "before Increment: ");
        printf("%p \n", ptr1);

        // Incrementing pointer ptr1;
        ptr1++;

        printf("Pointer ptr1 after"
                " Increment: ");
        printf("%p \n\n", ptr1);

        printf("Pointer ptr1 before"
                " Decrement: ");
        printf("%p \n", ptr1);

        // Decrementing pointer ptr1;
        ptr1--;

        printf("Pointer ptr1 after"
                " Decrement: ");
        printf("%p \n\n", ptr1);
```

```
    return 0;
}
```

Pointer Comparisons

Pointers may be compared by using relational operators, such as ==, <, and >. If p1 and p2 point to variables that are related to each other, such as elements of the same array, then p1 and p2 can be meaningfully compared.

The following program modifies the previous example − one by incrementing the variable pointer so long as the address to which it points is either less than or equal to the address of the last element of the array, which is &var[MAX - 1]

10.11 Pointer and Arrays

When an array is declared, the compiler allocates a base address and sufficient amount of storage to contain all the elements of the array in contiguous memory locations. The base address is the location of the first element (index 0) of the array. The compiler also defines the array name as a constant pointer to the first element.

The array declared as:

staticint x[5] = {1, 2, 3, 4, 5}; is stored as follows:

Elements x[0] x[1] x[2] x[3] x[4]

Value 1 2 3 4 5

Address 1000 1002 1004 1006 1008

The name x is defined as a constant pointer pointing to the first element, x[0] and therefore the value of x is 1000, the location where x[0] is stored. That is,

x = &x[0] = 1000

If we declare p as an integer pointer, then we can make the pointer p to point to the array x by the

assignment statement

p = x ;

which is equivalent to

p = &x[0];

Now we can access every value of x using p++ to move from one element to another. The relationship between p and x is shown below:

p = &x[0] (=1000)

p+1 = &x[1] (=1002)

p+2 = &x[2] (=1004)

p+3 = &x[3] (=1006)

The address of an element is calculated using its index and the scale factor of the data type, i.e., Address of x[3] = Base Address + (3 × Scale Factor of int) = 1000 + (3 × 2) = 1006

When handling arrays, instead of using array indexing, we can use pointers to access arrayelements, as *(p+3) gives the value of x[3]. The pointer accessing method is much faster than array indexing. &x[i] and (x+i) both represent the address of the ith element of x. x[i] and *(x+i) both represent the contents of that address, the value of the ith element of x. The two terms are interchangeable.

When assigning a value to an array element such as x[i], the left side of the assigned statement may be written as either x[i] or as *(x+i). Thus, a value may be assigned directly to an array element, or it may be assigned to the memory area whose address is that of the array element. While assigning an address to an identifier, a pointer variable must appear on the left side of the assignment statement.

Expressions such as x, (x+1) and &x[i] cannot appear on the left side of an assignment statement because it is not possible to assign an arbitrary address to an array name or an array element.

10.12 Array of Pointers

A multi-dimensional array can be expressed in terms of an array of pointers rather than as a pointer to a group of contiguous arrays. In such situations the newly defined array will have one less dimension than the original multi-dimensional array. Each pointer will indicate the beginning of a separate (n - 1) dimensional array.

In general terms, a two dimensional array can be defined as one dimensional array of pointers by writing

data_type *array[expression1];

rather than the conventional array definition data_type array[expression1] [expression2]; Similarly, a n dimensional array can be defined as a (n-1) dimensional array of pointers by writing

data_type *array[expression1][expression2]...[expressionn-1];

rather than the conventional array definition data_type array[expression1] [expression2]...

[expressionn];

In these declarations data_type refers to the data type of the original n dimensional array, array is the array name, and expression1, expression2, . . ., expression n are positive-valued integer expressions that indicate the maximum number of elements associated with each subscript.

The array name and its preceding asterisk are not enclosed in parentheses in this type of declaration. Thus, a right-to-left rule first associates the pairs of square brackets with array, defining the named object as an array. The preceding asterisk then establishes that the array will contain pointers.

Moreover, note that the last (the rightmost) expression is omitted when defining an array of pointers, whereas the first (the leftmost) expression is omitted when defining a pointer to a group of arrays.

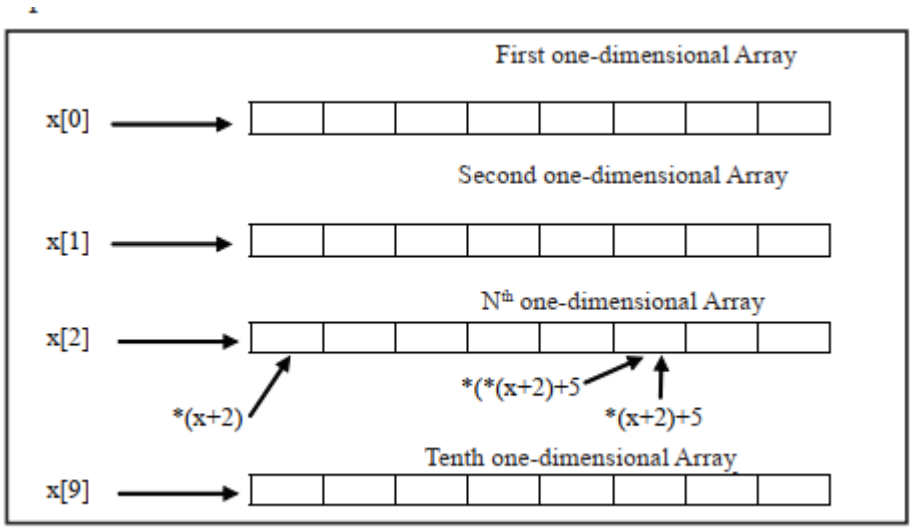
When a n dimensional array is expressed in this manner, an individual array element within the n dimensional array can be accessed by a single use of the indirection operator. The following example illustrates how this is done.

Suppose that x is a two dimensional integer array having 10 rows and 20 columns, we can define x as a one dimensional array of pointers by writing int *x[10];

Hence, x[0] points to the beginning of the first row, x[1] points to the beginning of the second row, and so on. The number of elements within each row is not explicitly specified.

An individual array element, such as x[2][5], can be accessed by writing *(x[2] + 5). In this expression, x[2] is a pointer to the first element in row 2, so that (x[2] + 5) points to element 5 (actually, the sixth element) within row 2. The object of this pointer, *(x[2] + 5), therefore, refers to x[2] [5].

These relationships are illustrated below:



10.13 Pointers and functions

We can use function pointers to avoid code redundancy. For example a simple qsort() function can be used to sort arrays in ascending order or descending or by any other order in case of array of structures. Not only this, with function pointers and void pointers, it is possible to use qsort for any data type.

C programming allows passing a pointer to a function. To do so, simply declare the function parameter as a pointer type. Following is a simple example where we pass an unsigned long pointer to a function and change the value inside the function which reflects back in the calling function.

```
#include <stdio.h>

#include <time.h>


voidgetSeconds(unsigned long *par);


int main () {


unsigned long sec;

getSeconds(&sec );


  /* print the actual value */

printf("Number of seconds: %ld\n", sec );


return 0;

}


voidgetSeconds(unsigned long *par) {

  /* get the current number of seconds */

  *par = time( NULL );

return;

}
```

Lab Exercise

```
// Program to demonstrate working of Pointers


#include<stdio.h>
int main(){

int n=123;
int *ptr=&n;
int **nn=n;

//*ptr=&n;
printf("Original value of variable n is = %d\n",n);
printf("Address of n is = %p\n",ptr);
printf("Address of n in decimal number is = %d\n",ptr);
printf("Value of  %d",nn);
return 0;
}
```

Lab Exercise

```
// Program to access value using Pointers


#include<stdio.h>
int main(){
int x;
printf("Enter Value of x\n");
scanf("%d",&x);
int *p;
p=&x;
printf("value of x entered by user is %d\n",x);
printf("Address of  variable x is %p\n",p);
printf("Getting value from pointer variable  %d\n",*p);
printf("Address of variable x is %d\n",p);
printf("Address of variable x is %u\n",p);
*p=500;
printf("New value of x is %d\n",x);
return 0;

}
```

Lab Exercise

```
// Program for Pointer Arithmatics


#include<stdio.h>
int main(){
inta,b,sum,sub,mul,div;
int *ptr1,*ptr2;
printf("Enter first number");
scanf("%d",&a);
printf("Enter second number");
```

```
scanf("%d",&b);
ptr1=&a;
ptr2=&b;
sum=*ptr1+*ptr2;
sub=*ptr1-*ptr2;
mul=*ptr1* *ptr2;
div= *ptr1/ *ptr2;
printf("Using pointers, all arithmetic operations
performed\n");
printf("Sum of first and second number is =%d\n",sum);
printf("Subtraction of first and second number is %d\n",sub);
printf("Product of first and second number is %d\n",mul);
printf("Division of first and second number is %d\n",div);
printf("Address of first and second number a=%p, b=%p
",ptr1,ptr2);
return 0;
}
```

10.14 NULL Pointer

A Null Pointer is a pointer that does not point to any memory location. It stores the base address of the segment. The null pointer basically stores the Null value while void is the type of the pointer.

If we do not have any address which is to be assigned to the pointer, then it is known as a null pointer. When a NULL value is assigned to the pointer, then it is considered as a **Null pointer**.

```
#include<stdio.h>
int main(){

int *ptr;
ptr=NULL;
printf("Value of null pointer is %d",ptr);
return 0;
}
```

Summary

- Pointers are often passed to a function as arguments by reference. This allows data items within the calling function to be accessed, altered by the called function, and then returned to the calling function in the altered form.
- There is an intimate relationship between pointers and arrays as an array name is really a pointer to the first element in the array.
- Access to the elements of array using pointers is enabled by adding the respective subscript to the pointer value (i.e. address of zeroth element) and the expression preceded with an indirection operator.
- As pointer declaration does not allocate memory to store the objects it points at, therefore, memory is allocated at run time known as dynamic memory allocation.
- The library routine malloc can be used for this purpose.

Keywords

Array of Pointer: A multi-dimensional array can be expressed in terms of an array of pointers rather than as a pointer to a group of contiguous arrays.

Pointer: It is a variable which can hold the address of a memory location rather than the value at the location.

Pointer Expression: Like other variables, pointer variables can be used in expressions. Arithmetic and comparison operations can be performed on the pointers

Self Assessment

1. What are the correct statements about pointers?

A. pointer is a variable that stores the address of another variable
B. pointer can also be used to refer to another pointer function
C. Pointers assign and releases the memory as well
D. all of above.

2. What are the applications of pointers?

A. Implement data structure
B. Dynamic memory allocation
C. Accessing array and functions
D. Above all

3. Which symbol is used during pointer declaration?

A. +
B. -
C. *
D. /

4. What format specifier is used for pointers?

A. %c
B. %d
C. %p
D. %s

5. Which statements are true about NULL pointers

A. NULL pointer pointing to nothing
B. The value of null pointer is 0
C. Both a and b
D. None of above

6. What is the output of following program?

```
#include<stdio.h>
int main(){
int *ptr=NULL;
printf("Value of null pointer is= %d",ptr);
return 0;
```

```
}
```

A. 1
B. 2
C. 0
D. 4

7. Which one is incorrect statement?

A. int x=90;
B. int *ptr1,*ptr2;
C. ptr1=@x;
D. ptr2=ptr1;

8. What type of arithmetic operations can performed on pointers?

A. Addition
B. Subtraction
C. Multiply
D. Above all

9. What is the output of following program?

```
#include<stdio.h>
int main(){
int x=90,y=10,result;
int *ptr1,*ptr2;
    ptr1=&x;
    ptr2=&y;
result=*ptr1**ptr2;
printf("Product of x and y  using pointers  is %d\n",result);
return 0;
}
```

A. 100
B. 80
C. 900
D. 10

10. Which one is incorrect statement?

A. int a=10,*ptr;
B. ptr=/a;
C. ptr--;
D. above all

11. What are the different operations that can be performed on pointers?

A. Decrement
B. Addition
C. Subtraction

D. Above all

12. Before using a pointer variable

A. It should be declared
B. It should be initialized
C. It should be both declared and initialized
D. None of above

13. Which is correct statement for expression?

Int *ptr, p;

A. ptr is a pointer to integer, p is not an integer variable.
B. ptr and p, both are pointers to integer.
C. ptr is pointer to integer, p may or may not be.
D. ptr and p both are not pointers to integer.

14. Which is correct statement?

A. int ->ptr[3];
B. int *ptr[2];
C. intptr[*5];
D. None of above

15. Which is correct statement for array of pointers?

A. We can store pointers to an int or char or any other data type available.
B. We can also declare an array of pointers
C. Use a pointer to point to an array, and then we can use that pointer to access the array elements
D. Above all

Answers for Self Assessment

1.	D	2.	D	3.	C	4.	C	5.	C
6.	C	7.	C	8.	D	9.	C	10.	C
11.	D	12.	C	13.	A	14.	B	15.	D

Review Questions

1. Define 'Pointer'. List down the various advantages of using pointers in a C program.

2. How pointer are initialized and implemented in C? Write a program to explain the concept.

3. Explain with the help of a C program, the concept of Pointer Arithmetic in C.

4. How printer in C incorporates the concept of Arrays? Write a suitable program to demonstrate the concept.

5. Differentiate the followings:

(a) Pointer and arrays

(b) Pointer to a variable and pointer to a pointer

(c) Pointer and variable

(d) Value in a function and address in a function

6. Twenty-five numbers are entered from the keyboard into an array. Write a program to find out how many of them are positive, how many are negative, how many are even and how many odd.

7. Write a function to calculate the factorial value of any integer entered through the keyboard.

8. Write a function power (a, b), to calculate the value of a raised to b.

9.Explain pointers and functions with suitable example.

10.As pointer declaration does not allocate memory to store the objects it points at, therefore, Memory is allocated at run time known as dynamic memory allocation.

Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C

Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

Unit 11: Strings

CONTENTS

Objectives

After studying this unit, you will be able to:

- Explain strings
- Describe reading and writing strings
- Explain string handling functions

Introduction

Computers process a variety of data kinds in addition to numeric data. The data to be processed is frequently textual, such as words, names, and addresses. String type variables are used to store and process this type of data. There is no explicit string data type in C.

Character arrays, on the other hand, can be used to simulate the same thing. In this session, we'll look at strings and how to manipulate them in C.

11.1 Strings

In C, a string is defined as a collection of characters. The NULL character, which signifies the end of the string, is used to end each string. Any group of characters enclosed in double-quote marks is referred to as a string constant.

When characters in a string constant are stored, the NULL character is automatically appended to the end of them. The escape sequence '0' is used to represent the NULL character within a

programme. A string constant is an array with a lower bound of 0 and an upper bound of the string's length in characters.

When choosing a data representation for a given data object, the cost of performing various operations with that representation must be considered. Furthermore, a hidden cost resulting from the required storage management procedures must be considered.

Strings are stored in three types of structures:

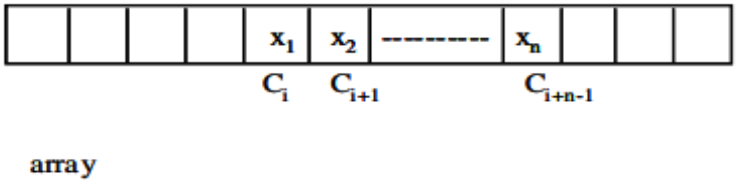
1. Fixed length structure

2. Variable length structure

3. Linked structure

9.2 Sequential Fixed Length Structure

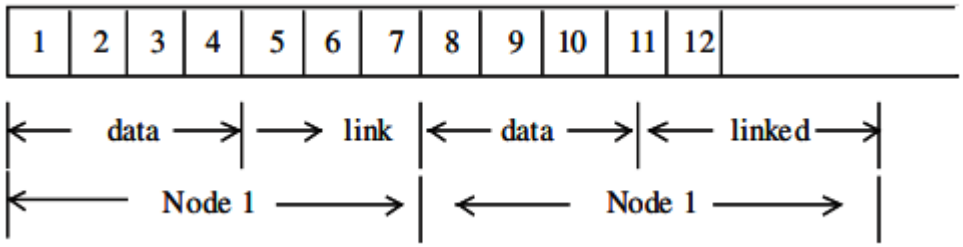
In this representation successive characters of a string will be placed in consecutive character positions. The string S = 'x1---xn' could then be represented as in Figure with s as a pointer to the first character.



Now, if we want to pick a substring of size k from the string of size n, the time required to achieve this would be O(k) plus the time needed to locate a free space big enough to hold the string.

Linked List Fixed Size Nodes

The available memory is divided into nodes of fixed size. Each node has two fields: Data andLink. The size of a node is number of characters that can be stored in the DATA fields.



In the above figure memory is divided into nodes of size 4 with a link field that is two characters long. Deletion of a substring can be carried out by replacing all characters in this substring by 0 and freeing nodes in which the data fields consist of only 0's.

Storage compaction can be carried out when there are no free nodes. String representation with variable size is similar.

Each node in the purest version of a linked list representation of strings would be one in size. Normally, this would be considered a huge waste of space. With a two-character link field, this means that only 1/3 of the available Memory will be used to store string data, while the remaining 2/3 will be used exclusively for link data.

11.2 Declaring and Initializing String

In C, strings are represented as character arrays. The null character, which is just the character with the value 0, is used to indicate the end of the string. (The null character is unrelated to the null pointer except by name.) The null character is known as NUL in the ASCII character set.) Another character escape sequence, 0 represents the null or string-terminating character.

Because C has no built-in facilities for manipulating entire arrays (copying them, comparing them, etc.), it also has very few built-in facilities for manipulating strings.

In fact, C's only truly built-in string-handling is that it allows us to use string constants (also called string literals) in our code. Whenever we write a string, enclosed in double quotes, C automatically creates an array of characters for us, containing that string, terminated by the \0 character.

: We can declare and define an array of characters, and initialize it with a stringconstant:

char string[] = "Hello, world!";

In this situation, we may omit the array's dimension because the compiler will figure it out for us based on the size of the initializer. The compiler will only size a string array for us in this scenario; in all other circumstances, we will have to decide how big the arrays and other data structures we employ to house strings are

To do anything else with strings, we must typically call functions. The C library contains a fewbasic string manipulation functions, and to learn more about strings, we'll be looking at howthese functions might be implemented.

Since C never lets us assign entire arrays, we use the strcpy function to copy one string toanother:

```
#include <string.h>

char string1[] = "Hello, world!";

char string2[20];

strcpy(string2, string1);
```

The destination string is strcpy's first argument, so that a call to strcpy mimics an assignment expression (with the destination on the left-hand side). Notice that we had to allocate string2 big enough to hold the string that would be copied to it. Also, at the top of any source file where we're using the standard library's string-handling functions (such as strcpy) we must include the line

```
#include <string.h>
```

which contains external declarations for these functions.

Since C won't let us compare entire arrays, either, we must call a function to do that, too. The standard library's strcmp function compares two strings, and returns 0 if they are identical, or a negative number if the first string is alphabetically "less than'' the second string, or a positive number if the first string is "greater." (Roughly speaking, what it means for one string to be "less than'' another is that it would come first in a dictionary or telephone book, although there are a few anomalies.) Here is an example:

```
char string3[] = "this is";

char string4[] = "a test";

if(strcmp(string3, string4) == 0)

printf("strings are equal\n");

elseprintf("strings are different\n");
```

This code fragment will print "strings are different''. Notice that strcmp does not return a Boolean,

true/false, zero/nonzero answer, so it's not a good idea to write something like

```
if(strcmp(string3, string4))

...
```

because it will behave backwards from what you might reasonably expect. (Nevertheless, if you start reading other people's code, you're likely to come across conditionals like if(strcmp(a, b)) or even if(!strcmp(a, b)). The first does something if the strings are unequal; the second does something if they're equal. You can read these more easily if you pretend for a moment that strcmp's name were strdiff, instead.)

Another standard library function is strcat, which concatenates strings. It does not concatenate two strings together and give you a third, new string; what it really does is append one string onto the

end of another. (If it gave you a new string, it would have to allocate memory for it somewhere, and the standard library string functions generally never do that for you automatically.) Here's an example:

```
char string5[20] = "Hello, ";
char string6[] = "world!";
printf("%s\n", string5);
strcat(string5, string6);
printf("%s\n", string5);
```

The first call to printf prints ``Hello, '', and the second one prints "Hello, world!", indicating that the contents of string6 have been tacked on to the end of string5. Notice that we declared string5 with extra space, to make room for the appended characters.

If you have a string and you want to know its length (perhaps so that you can check whether it will fit in some other array you've allocated for it), you can call strlen, which returns the length of the string (i.e. the number of characters in it), not including the \0:

```
char string7[] = "abc";
intlen = strlen(string7);
printf("%d\n", len);
```

Finally, you can print strings out with printf using the %s format specifier, as we've been doingin these examples already (e.g. printf("%s\n", string5);).

Since a string is just an array of characters, all of the string-handling functions we've just seen can be written quite simply, using no techniques more complicated than the ones we already know. In fact, it's quite instructive to look at how these functions might be implemented. Here is a version of strcpy:

```
mystrcpy(char dest[], char src[])
{
inti = 0;
while(src[i] != '\0')
{
dest[i] = src[i];
i++;
}
dest[i] = '\0';
}
```

We've called it mystrcpy instead of strcpy so that it won't clash with the version that's already in the standard library. Its operation is simple: it looks at characters in the src string one at a time, and as long as they're not \0, assigns them, one by one, to the corresponding positions in the dest string. When it's done, it terminates the dest string by appending a \0. (After exiting the while loop, i is guaranteed to have a value one greater than the subscript of the last character in src.) For comparison, here's a way of writing the same code, using a for loop:

```
for(i = 0; src[i] != '\0'; i++)
dest[i] = src[i];
dest[i] = '\0';
```

Yet a third possibility is to move the test for the terminating \0 character out of the for loop header and into the body of the loop, using an explicit if and break statement, so that we can perform the test after the assignment and therefore use the assignment inside the loop to copy

the \0 to dest, too:

```
for(i = 0; ; i++)
```

```
{
dest[i] = src[i];
if(src[i] == '\0')
break;
}
```

(There are in fact many, many ways to write strcpy. Many programmers like to combine the assignment and test, using an expression like (dest[i] = src[i]) != '\0'. Here is a version of

```
strcmp:
mystrcmp(char str1[], char str2[])
{
inti = 0;
while(1)
{
if(str1[i] != str2[i])
return str1[i] - str2[i];
if(str1[i] == '\0' || str2[i] == '\0')
return 0;
i++;
}
}
```

Characters are compared one at a time. If two characters in one position differ, the strings are different, and we are supposed to return a value less than zero if the first string (str1) is alphabetically less than the second string. Since characters in C are represented by their numeric character set values, and since most reasonable character sets assign values to characters in alphabetical order, we can simply subtract the two differing characters from each other: the expression str1[i] - str2[i] will yield a negative result if the i'th character of str1 is less than the corresponding character in str2. (As it turns out, this will behave a bit strangely when comparing upper- and lower-case letters, but it's the traditional approach, which the standard versions of strcmp tend to use.) If the characters are the same, we continue around the loop, unless the characters we just compared were (both) \0, in which case we've reached the end of both strings, and they were both equal. Notice that we used what may at first appear to be an infinite loop—the controlling expression is the constant 1, which is always true. What actually happens is that the loop runs until one of the two return statements breaks out of it (and the entire function).

Finally, here is a version of strlen:

```
intmystrlen(char str[])
{
inti;
for(i = 0; str[i] != '\0'; i++)
{}
returni;
}
```

In this case, all we have to do is find the \0 that terminates the string, and it turns out that the three control expressions of the for loop do all the work; there's nothing left to do in the body.

Therefore, we use an empty pair of braces {} as the loop body. Equivalently, we could use a null statement, which is simply a semicolon:

```
for(i = 0; str[i] != '\0'; i++)
```

Empty loop bodies can be a bit startling at first, but they're not unheard of.Everything we've looked at so far has come out of C's standard libraries. As one last example,let's write a substr function, for extracting a substring out of a larger string. We might call it likethis:

```
char string8[] = "this is a test";
char string9[10];
substr(string9, string8, 5, 4);
printf("%s\n", string9);
```

The idea is that we'll extract a substring of length 4, starting at character 5 (0-based) of string8, andcopy the substring to string9. Just as with strcpy, it's our responsibility to declare the destinationstring (string9) big enough. Here is an implementation of substr. Not surprisingly, it's quite similar to strcpy:

```
substr(char dest[], char src[], int offset, intlen)
{
inti;
for(i = 0; i<len&&src[offset + i] != '\0'; i++)
dest[i] = src[i + offset];
dest[i] = '\0';
}
```

If you compare this code to the code for mystrcpy, you'll see that the only differences are thatcharacters are fetched from src[offset + i] instead of src[i], and that the loop stops when Len characters have been copied (or when the src string runs out of characters, whichever comesfirst).

In this unit, we've been careless about declaring the return types of the string functions, and(with the exception of mystrlen) they haven't returned values. The real string functions do returnvalues, but they're of type ``pointer to character,'' which we haven't discussed yet.When working with strings, it's important to keep firmly in mind the differences betweencharacters and strings. We must also occasionally remember the way characters are represented,and about the relation between character values and integers.

As we have had several occasions to mention, a character is represented internally as a smallinteger, with a value depending on the character set in use. For example, we might find that'A' had the value 65, that 'a' had the value 97, and that '+' had the value 43. (These are, in fact,the values in the ASCII character set, which most computers use. However, you don't need tolearn these values, because the vast majority of the time, you use character constants to referto characters, and the compiler worries about the values for you. Using character constants inpreference to raw numeric values also makes your programs more portable.)

As we may also have mentioned, there is a big difference between a character and a string, evena string which contains only one character (other than the \0).

'A' is not the same as "A". To drive home this point, let's illustrate it with a fewexamples.

If you have a string:

```
char string[] = "hello, world!";
```

you can modify its first character by saying

```
string[0] = 'H';
```

(Of course, there's nothing magic about the first character; you can modify any character in the string in this way. Be aware, though, that it is not always safe to modify strings in-place like this; we'll say more about the modifiability of strings in a later unit on pointers.) Since you're replacing a character, you want a character constant, 'H'. It would not be right to write

```
string[0] = "H"; /* WRONG */
```

because "H" is a string (an array of characters), not a single character. (The destination of the assignment, string[0], is a char, but the right-hand side is a string; these types don't match.) On the other hand, when you need a string, you must use a string. To print a single newline, you could call

printf("\n");

It would not be correct to call

printf('\n'); /* WRONG */

printf always wants a string as its first argument. (As one final example, putchar wants a single character, so putchar('\n') would be correct, and putchar("\n") would be incorrect.) We must also remember the difference between strings and integers. If we treat the character '1' as an integer, perhaps by saying

inti = '1';

we will probably not get the value 1 in i; we'll get the value of the character '1' in the machine's character set. (In ASCII, it's 49.) When we do need to find the numeric value of a digit character (or to go the other way, to get the digit character with a particular value) we can make use of the fact that, in any character set used by C, the values for the digit characters, whatever they are, are contiguous. In other words, no matter what values '0' and '1' have, '1' - '0' will be 1 (and, obviously, '0' - '0' will be 0). So, for a variable c holding some digit character, the expression

c - '0'

gives us its value. (Similarly, for an integer value i, i + '0' gives us the corresponding digit character, as long as 0 <= i<= 9.)

Just as the character '1' is not the integer 1, the string "123" is not the integer 123. When we have a string of digits, we can convert it to the corresponding integer by calling the standard function

atoi:

char string[] = "123";

inti = atoi(string);

int j = atoi("456");

11.3 Reading and Writing Strings

Character String Input Function

% ws or % wc can be used as the specification for reading character strings. The specifier % terminates reading a string at the encounter of blank space. Some versions of scanf( ) support the following conversion specification for strings.

% [characters] and % [^characters]

The specification % [characters] means that only the characters specified within the brackets are permissible in the input string. If the input string contains any other character, the string will be terminated at the first encounter of such a character.

The specification % [^character] does exactly the reverse, i.e., characters specified after circumflex (^) are not permitted.

gets( ) function is used to read a character entered at the keyboard and places it at the address pointed to by its character pointer argument.

Characters are entered until the enter key is pressed.

syntax: char * gets (char *a);

where a is the character array.

Character String Output Function

puts( ) function writes its string argument to the screen followed by the new line.

syntax: char * puts (const char * a);

puts( ) function takes less space thenprintf( ). It is faster than printf( ). It does not output numbers

or does format conversions as puts( ) outputs character string only.

include <stdio.h>

```
# include <conio.h>
main( )
{
charstr [50]
gets (str);
puts (str);
}
```

11.4 Build-in-Library Functions to Manipulate Strings

With every C compiler a large set of useful string handling library functions are provided.

Table lists the more commonly used functions along with their purpose.

Functions	Use
strlen	Finds length of a string
strlwr	Converts a string to lowercase
strupr	Converts a string to uppercase
strcat	Appends one strings to uppercase
strncat	Appends first n characters of a string at the end of another
strcpy	Copies a string into another
strncpy	Copies first n characters o one string into another
strcmp	Compares two strings
strncmp	Compares first n characters of two strings
strcmpi	Compares two strings without regard to case ("I" denotes that this function ignores case)
stricmp	Compares two strings without regards to case (identical to strcmpi)
strnicmp	Compares first n characters if two strings without regard to case
strdup	Duplicates a string
strchr	Finds first occurrence of a given character in a string
strchr	Finds last occurrence of a given character in a string
strrchr	Finds last occurrence of a given character in a string
strstr	Finds first occurrence of a given striung in another string
strset	Sets all characers of string to a given character
strnset	Sets first n characters of a string to a given character
strrev	Reverses string

Out of the above list, We shall discuss the functions strlen( ), strcpy( ), strcat( ) and strcmp( ), since these are the most commonly used functions. This will also illustrate how the library functions in general handle strings. Let us study these functions one by one.

11.5 strlen( )

This function counts the number of characters present in a string. Its usage is illustrated in the following program.

```
main( )
{
chararr[ ] = "Bamboozled" ;
int len1, len2 ;
```

```
len1 = strlen( arr ) ;
len2 = strlen( "Humpty Dumpty" ) ;
printf ( "\nstring = %s length = %d", arr, len1 ) ;
printf ( "\nstring = %s length = %d", "Humpty Dumpty", len2 ) ;
}
```

The output would be...

string = Bamboozled length = 10

string = Humpty Dumpty length = 13

11.6 strcpy( )

This function copies the contents of one string into another. The base addresses of the source and target strings should be supplied to this function. Here is an example of strcpy( ) in action...

```
main( )
{
char source[ ] = "Sayonara" ;
char target[20] ;
strcpy ( target, source ) ;
printf ( "\nsource string = %s", source ) ;
printf ( "\ntarget string = %s", target ) ;
}
```

And here is the output...

source string = Sayonara

target string = Sayonara

On supplying the base addresses, strcpy( ) goes on copying the characters in source string into the target string till it doesn't encounter the end of source string ('\0'). It is our responsibility to see to it that the target string's dimension is big enough to hold the string being copied into it.

Thus, a string gets copied into another, piece-meal, character by character. There is no short cut for this. Let us now attempt to mimic strcpy( ), via our own string copy function, which we will call xstrcpy( ).

```
main( )
{
char source[ ] = "Sayonara" ;
char target[20] ;
xstrcpy ( target, source ) ;
printf ( "\nsource string = %s", source ) ;
printf ( "\ntarget string = %s", target ) ;
}
xstrcpy ( char *t, char *s )
{
while ( *s != '\0' )
{
```

```
        *t = *s ;
        s++ ;
        t++ ;
        }
        *t = '\0' ;
}
```

The output of the program would be...

source string = Sayonara

target string = Sayonara

11.7 strcat( )

This function concatenates the source string at the end of the target string. For example, "Bombay"and "Nagpur" on concatenation would result into a string "BombayNagpur". Here is an example of strcat( ) at work.

```
#include<stdio.h>
#include<string.h>
int main(){
char name[10];
charlname[10];
printf("Enter Name");
gets(name);
printf("Enter last name");
gets(lname);
strcat(name,lname);
printf("Full name is %s",name);
return 0;

}
```

11.8 strcmp( )

This is a function which compares two strings to find out whether they are same or different. The two strings are compared character by character until there is a mismatch or end of one of the strings is reached, whichever occurs first. If the two strings are identical, strcmp( ) returns a value zero. If they're not, it returns the numeric difference between the ASCII values of the first non-matching pairs of characters. Here is a program which puts strcmp( ) in action.

```
#include<stdio.h>
 #include<string.h>
int main(){

 char str1[10]="aBC";
char str2[10]="ABC";
```

```
printf("Comparing Str1 and Str2  = %d",strcmp(str1,str2));
return 0;
}
```

11.9 Putting String Together

```
#include <string.h>
#include <stdio.h>
int main()
{
char first[100];
char last[100];
charfull_name[200];
strcpy(first, "firstName");
strcpy(last, "secondName");
strcpy(full_name, first);
strcat(full_name, " ");
strcat(full_name, last);
printf("The full name is %s\n", full_name);
return (0);
}
```

11.10 Comparison of two String

```
#include <string.h>
i = strcmp( s1, s2 );
Where:
const char *s1, *s2;
are the strings to be compared.
inti;
```

gives the results of the comparison. "i" is zero if the strings are identical. "i" is positive if string "s1" is greater than string "s2", and is negative if string "s2" is greater than string "s1". Comparisons of "greater than" and "less than" are made according to the ASCII collating sequence.

11.11 String Handling Functions

A close analysis of the essential string-handling facilities required of any text creation and editing system (formal or otherwise) should lead to the following list of primitive functions:

1. Create a string of test
2. Concatenate two strings to form another string
3. Search and replace (if desired) a given substring within a string
4. Test for the identity of a string
5. Compute the length of a string

String related functions are grouped into string.h header file. It contains the following functions among others:

1. char * strcat(char *dest, const char *src): This function appends one string to another returning a pointer to concatenated string. It appends a copy of src to the end of dest. The length of the resulting string is strlen(dest) + strlen(src). strings.

2. intstrcmp(const char *s1, const char *s2): This function compares two strings. The string comparison starts with the first character in each string and continues with subsequent characters until the corresponding characters differ or until the end of the strings is reached.

The returned values are integers as follows:

< 0 if s1 < s2

= 0 if s1 = = s2

> 0 if s1 > s2

3. char *strcpy(char *dest, const char *src): This function copies string src to dest stopping after the terminating null character has been moved. The return value is dest.

4. intstrlen(const char *s): This function returns the length of a string (i.e., the number of characters in s), not counting the terminating null character.

5. intstrncmp(const char *s1, const char *s2, intmaxlen): This function compares portions of two strings s1 and s2 looking at no more than maxlen characters. The string comparison starts with the first character in each string and continues with subsequent characters until the corresponding characters differ or until maxlen characters have been examined. It returns an int value based on the result of comparing s1 (or part of it) to s2 (or part of it) as

given below:

< 0 if s1 < s2

= 0 if s1 == s2

> 0 if s1 > s2

11.12 Pointers and Strings

A String is a sequence of characters stored in an array. A string always ends with null ('\0') character.

A pointer to array of characters or a string can hold value and address both.

Syntax

char *names[];

```
#include <stdio.h>
int main()
{
char *names[] = {"ABC", "DEF"};
inti;
for(i = 0; i< 2; i++)
printf("%s \n", names[i]);
return 0;
}
```

Summary

- A string is defined in C as an array of characters. Each string is terminated by the NULL character, which indicates end of the string.
- A string constant is denoted by any set of characters included in double-quote marks.
- The NULL character is automatically appended to the end of the characters in a string constant when they are stored. Within a program, the NULL character is denoted by the escape sequence '\ 0'.
- A string constant represents an array whose lower bound is 0 and whose upper bound is the number of characters in the string.
- Strings are stored in three types of structures - Fixed length structure, Variable length structure, and Linked structure.

Keywords

gets( ): A C library function used to read a character entered at the keyboard and to place it at theaddress pointed to by its character pointer argument

puts( ): A C library function that writes its string argument to the screen followed by the newline

strcat(): The C library function that appends one string to another returning a pointer to concatenated string

strcmp(): The C library function that compares two strings

string.h: A C header file that contains string manipulating library functions

String: An array of characters terminated by the NULL character

Self Assessment

1. Which one is correct method for Initializing string?

A. char abc[ ] ="hello";
B. char abc[10]= {'h','e','l','l','o','\0'};
C. Char abc[ ] = {'h','e','l','l','o','\0'};
D. above all

2. Which method is use to read string ?

A. Puts ( )
B. Gets ( )
C. Print ( )
D. None of above

3. Which method is use to write string?

A. Scanf ( )
B. Gets ( )
C. Puts ( )
D. Printf ( )

4. String is an_____

A. an array of characters

B. sequence of characters terminated with a null character
C. both a and b
D. none of above

5. Which of the following function is more appropriate for reading in a multi-word string?

A. gets ( )
B. Puts ( )
C. Printf ( )
D. Sizeo ( )

6. Format specifier is used to print a string is _____

A. % d
B. % c
C. % s
D. % f

7. Is there any difference between the two statements?

char *ch = "string ";

charch[] = "String";

A. Yes
B. No

8. Function of strcat ( ) is_____

A. compares two strings
B. converts string to lowercase
C. concatenates (joins) two strings
D. none of above

9. If the two strings are identical, then strcmp( ) function returns

A. 1
B. 0
C. 2
D. -1

10. The library function used to computes string's length

A. strlwr ( )
B. strcpy ( )
C. strlen ( )
D. strpr ( )

11. What will strupr( ) function do?

A. compares two strings
B. converts string to uppercase

C. computes string's length

D. none of above

12. Function of strcpy ( ) is_____

A. converts string to uppercase

B. sequence of characters terminated with a null character

C. copies a string to another

D. none of above

13. What is the maximum length of a C String?

A. 16 characters

B. 8 characters

C. 32 characters

D. None of above

14. What will strlwr( ) function do?

A. converts string to lowercase

B. converts string to uppercase

C. computes string's length

D. none of above

15. Find incorrect statement _____

A. char input[100];

B. puts('Input string');

C. gets(input);

D. puts("Entered string is");

Answers for Self Assessment

1.	D	2.	B	3.	C	4.	C	5.	A
6.	C	7.	A	8.	C	9.	B	10.	C
11.	B	12.	C	13.	D	14.	A	15.	B

Review Questions

1. Write a C program that reads a sentence from the keyboard and prints the frequency of each letter.

2. How can you create a string type C variable? Can they be assigned to each other in the same way as other data types? Explain.

3. Write a program that converts a string like "124" to an integer 124.

4. Write a program that replaces two or more consecutive blanks in a string by a single blank.

For example, if the input is